

Ej01.01

Describe brevemente limitaciones de las base de datos relacionales y con que características las base de datos NoSql mitigan este problema.

Algunas limitaciones que tenían las bases de datos relacionales en la época que empieza a surgir nosql son:

- Escalabilidad vertical vs. horizontal

Originariamente, para hacer crecer una base de datos relacional, era necesario aumentar la cantidad de recursos de la instancia, en lugar de poder agregar mayor cantidad de máquinas chiquitas.

vs NoSQL:

Están diseñadas para agregarse en clústeres de nodos iguales (“peers”) y repartir los datos automáticamente mediante particiones basadas en la clave (sharding). Al necesitar más capacidad, solo se añaden nodos al clúster en lugar de reforzar un único servidor

- Esquema rígido

Antes de insertar datos es necesario definir tablas, tipos de columnas y relaciones. Cualquier cambio en el esquema (añadir o quitar columnas, modificar tipos) suele requerir migraciones o bloqueos temporales

vs NoSQL:

Bases de datos de documento (MongoDB, CouchDB), clave-valor (Redis, Riak) o wide-column (Cassandra, HBase) permiten almacenar objetos con campos diferentes entre sí sin necesidad de migraciones formales. Si mañana se debe guardar un campo adicional se inserta directamente en el documento, sin alterar una definición global de tabla

- Operaciones JOIN y complejidad de consultas

Para relacionar varias tablas (por ejemplo, “clientes → pedidos → productos”), se usan JOINS que pueden volverse muy costosos cuando las tablas crecen a millones de filas

vs NoSQL:

En lugar de usar JOINS caros, muchos NoSQL fomentan la desnormalización y el diseño de “data models” basados en necesidades de consulta (por ejemplo, almacenar la colección de items de un carrito dentro del mismo documento de usuario). Al hacerlo, la aplicación lee un único documento o realiza consultas sencillas sin múltiples uniones relacionales

- Dificultad para manejar datos no estructurados o semiestructurados

Aunque hoy en día la mayoría de RDBMS admiten tipos JSON o XML, el modelo relacional original está diseñado para datos estrictamente tabulares. Al guardar documentos muy variados (por ejemplo, logs, artículos con distinto esquema, metadatos flexibles), se recurre a columnas tipo TEXT/JSON que no benefician de índices nativos tan eficaces como en NoSQL

vs NoSQL:

Consistencia eventual (BASE) frente a ACID estricto.

Muchas NoSQL optan por el modelo BASE (Basically Available, Soft state, Eventual consistency) en vez de ACID estricto. Eso permite replicar datos rápidamente entre nodos y aceptar temporalmente estados “no totalmente sincronizados”, garantizando que a medio plazo (milisegundos o segundos) todos los nodos converjan

- Dificultad en la replicación y particionado nativo

Muchas bases relacionales ofrecen réplicas (master–slave), pero el sharding (particionado horizontal) suele ser una capa extra que hay que gestionar manualmente (por ejemplo, particionar tablas según rangos de claves).

vs NoSQL:

Desde su arquitectura, muchas NoSQL proporcionan:

Replicación automática: mantienen copias de los datos en varios nodos para tolerancia a fallos.

Particionado (sharding) integrado: distribuyen datos según la clave o rangos y balancean automáticamente la carga.